

## DATA 260 Homework 5

### Open-Source Vulnerability Reporting Platform and MCP Tools

Student: Vrishin KunnathamParambath  
SID4: 8844

## 0. Configuration and Repository

| Item             | Value                     |
|------------------|---------------------------|
| SID4             | 8844                      |
| PORT_BASE        | 8744                      |
| PREFIX           | s8844                     |
| SEED             | 8844                      |
| VERIFY_SEED      | 268844                    |
| DOMAIN_ID        | 4                         |
| Operating system | Windows 11                |
| Hardware         | Local CPU environment     |
| Local model      | Ollama qwen3:8b           |
| Branch           | hw5-development           |
| Final commit     | UPDATE_AFTER_FINAL_COMMIT |

Repository: <https://github.com/Viki2183/data260-8844>

Homework branch: <https://github.com/Viki2183/data260-8844/tree/hw5-development>

Sbnikitha and supriyaselvangesan have access to the repository.

## 1. Database, API, and Redux Client

I extended the existing vulnerability-reporting application by adding a related `packages` entity. Each package can have multiple vulnerability reports. Package codes use the required `s8844-` format.

| Entity                             | Main fields                                                                                                                                                                                                                                                                                  |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>packages</code>              | <code>id</code> , <code>name</code> , <code>ecosystem</code> , <code>packageCode</code> , <code>createdAt</code> , <code>updatedAt</code>                                                                                                                                                    |
| <code>vulnerability_reports</code> | <code>id</code> , <code>packageId</code> , <code>packageName</code> , <code>vulnerabilityId</code> , <code>submitterEmail</code> , <code>vulnerabilityDescription</code> , <code>severity</code> , <code>affectedVersionsCount</code> , <code>termsAccepted</code> , <code>timestamps</code> |

The API supports CRUD operations for both entities and a relationship endpoint for retrieving reports belonging to a package. Validation covers email format, severity, required text, package-code format, positive IDs, description length, and terms acceptance. Invalid inputs return 422, missing resources return 404, and restricted deletes return 409.

| Area                            | Evidence      |
|---------------------------------|---------------|
| Package CRUD                    | Figures 7–15  |
| Vulnerability-report CRUD       | Figures 16–22 |
| Database                        | Figure 23     |
| Redux Home/Create/Update/Delete | Figures 24–27 |

The Redux client stores report data in a Redux Toolkit slice. Async thunks load, create, update, and delete reports while the UI displays loading and error states.

```
export const fetchReports = createAsyncThunk(
  "reports/fetchReports",
  async ({ search = "", skip = 0, limit = 100 } = {},
    { rejectWithValue }) => {
    try {
      return await listReports(search, skip, limit);
    } catch (error) {
      return rejectWithValue(getErrorMessage(error));
    }
  },
);
```

## 2A. TheMealDB MCP Server

The first MCP server provides four read-only meal tools. I tested each tool through MCP Inspector with real inputs and reviewed the returned data.

| Tool                 | Purpose                         |
|----------------------|---------------------------------|
| search_meals_by_name | Search meals by name.           |
| meals_by_ingredient  | Find meals using an ingredient. |
| random_meal          | Return one random meal.         |
| meal_details         | Return details for one meal.    |

Evidence: Figures 28–31.

## 2B. Domain MCP Server

The second MCP server operates on the s8844 package and vulnerability data. It exposes exactly three tools: search, detail, and aggregate.

```
{
  "ok": true,
  "data": {},
  "error": null
}
```

Successful calls return data and a null error. Rejected calls return `ok: false` with a clear error message. Datetime values are converted to strings before serialization, and server logs go to `stderr`.

Evidence: Figures 32–37.

### 3. Validation, Retries, and Fault Injection

---

The three rejected inputs were a blank search query, package ID zero, and the unsupported severity value `Extreme`. The retry policy allows two retries after the initial attempt and uses bounded exponential backoff.

| Failure rate | Calls | Success rate | Mean latency | P99 latency |
|--------------|-------|--------------|--------------|-------------|
| 0%           | 50    | 100%         | 1.602 ms     | 2.007 ms    |
| 20%          | 50    | 100%         | 1.587 ms     | 1.733 ms    |
| 50%          | 50    | 88%          | 1.433 ms     | 1.993 ms    |

The results show that retries recovered transient failures at 20%. At 50%, retries improved reliability but six calls still failed after the retry limit. This policy is appropriate for interactive use because it keeps delays bounded. Batch processing could use more retries and longer backoff.

Evidence: Figure 38 and the raw JSONL files in `reports/hw05/raw`.

### 4. Safe Tool Entry Point and Offline Tests

---

`execute_tool(name, inputs)` is the single entry point used by the agent. It validates tool names, catches errors, reuses the shared response envelope, and returns JSON instead of crashing.

The offline runner uses fixture handlers so it can be executed without an LLM or database. It produced 7/7 passing tests covering valid calls, rejected inputs, and an unknown tool.

Evidence: Figure 39.

### 5. Safety Rule and Ollama Agent

---

The safety rule blocks searches containing credential-like terms such as `password`, `secret`, `token`, or `credential`. A normal package search is allowed, while a credential-like search returns a structured blocked response without executing the tool.

The agent loop has a maximum step count. It records each request, selected tool, inputs, result, and final stop reason in `agent_runs.jsonl`.

| Test                        | Result |
|-----------------------------|--------|
| Allowed safety call         | PASS   |
| Blocked safety call         | PASS   |
| MockModel normal completion | PASS   |
| MockModel max_steps         | PASS   |

Four local Ollama scenarios were completed with `qwen3:8b`. CPU inference required a longer timeout. Disabling extended reasoning and limiting the response to JSON made the model responses more reliable.

Evidence: Figures 40–41.

### AI Use Disclosure

---

## What assistance was used?

I used an AI assistant to understand the assignment, plan the MCP and retry implementation, troubleshoot errors, and improve the report structure. I ran the commands, tested the application, reviewed the results, and captured the evidence myself.

## What went wrong?

The first MCP code used an MCP 1.x import while the environment initially had MCP 2.x. Ollama also timed out on CPU and sometimes returned incomplete JSON.

## How was it verified?

I used the Python traceback, checked the installed package version, reviewed MCP Inspector results, reran the offline tests, and reran the Ollama scenarios.

## What was changed?

I pinned MCP to a compatible version, corrected the server launch command, added datetime serialization, increased the Ollama timeout, disabled extended reasoning, and restricted the model response to JSON.

## Reflection

---

This homework connected a database-backed web application to MCP tools and a bounded local agent. The shared response envelope was one of the most useful design choices because successful results and rejected inputs followed the same structure. This made the offline tests easier to write and made errors easier to understand.

The retry experiment showed that retries help with temporary failures but cannot solve persistent failures. The 20% experiment achieved a 100% success rate, while the 50% experiment achieved 88%. For interactive use, a small retry budget keeps the application responsive. Batch processing could use more retries and longer delays.

The safety rule was also important. A model should not be able to search for credential-like data simply because it generated that request. The blocked-call test confirmed that safety is checked before tool execution. The maximum-step test prevented an endless tool loop. Finally, local Ollama inference showed that CPU execution may require longer timeouts and strict JSON output handling. These checks made the system more predictable than relying on the model alone.

## Evidence Screenshots

---

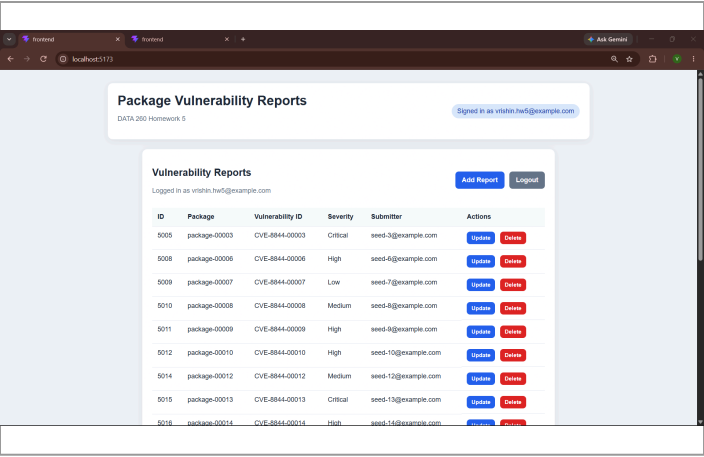


Figure 1: Home reports page

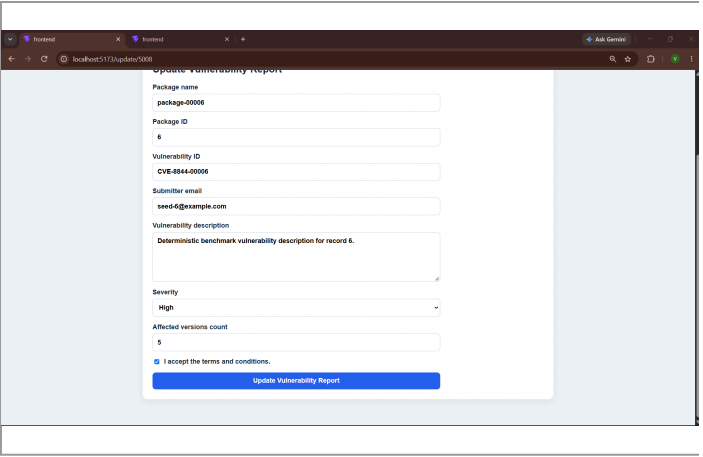


Figure 2: Update form

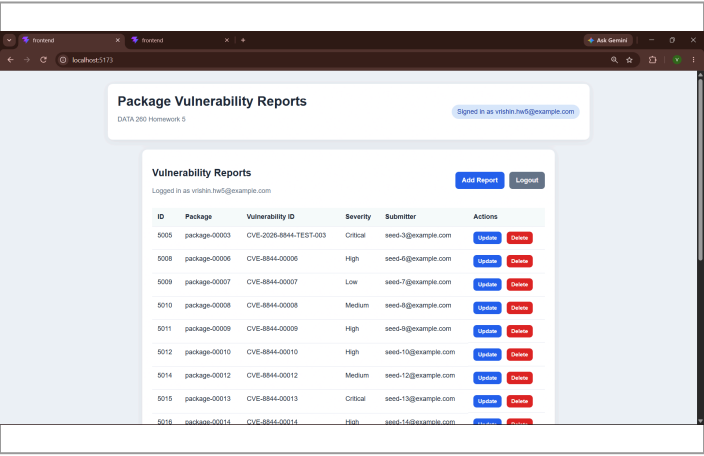


Figure 3: Created report

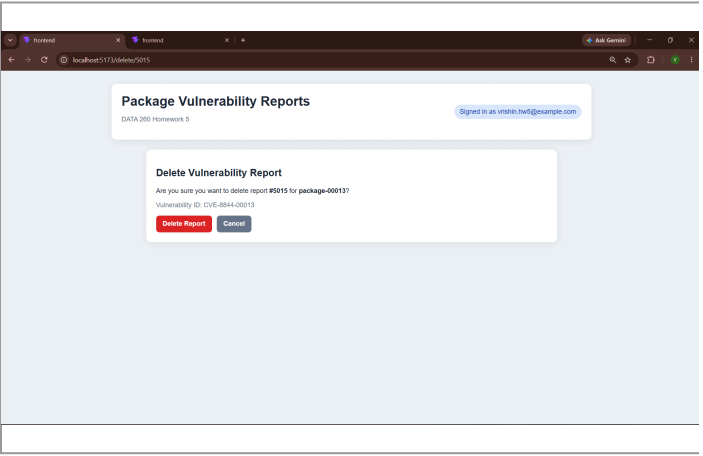


Figure 4: Delete confirmation

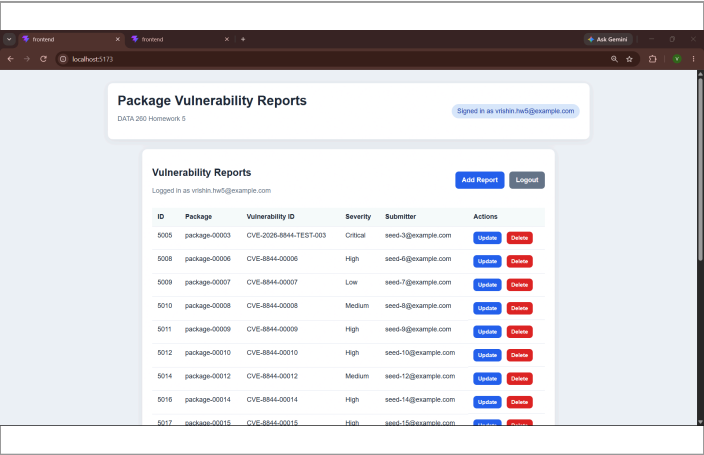


Figure 5: After deletion

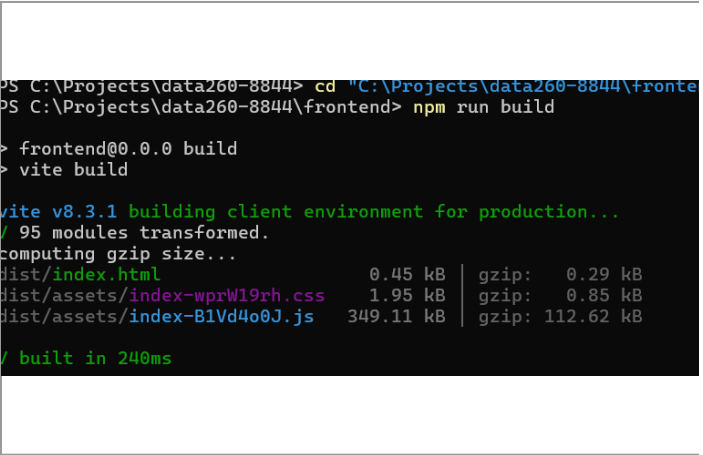
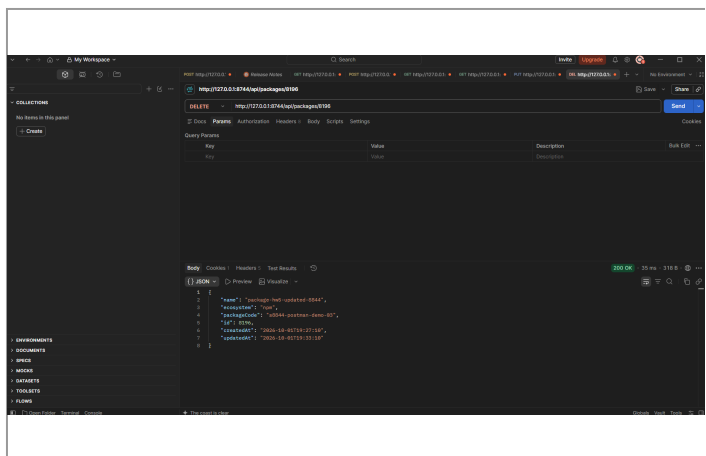
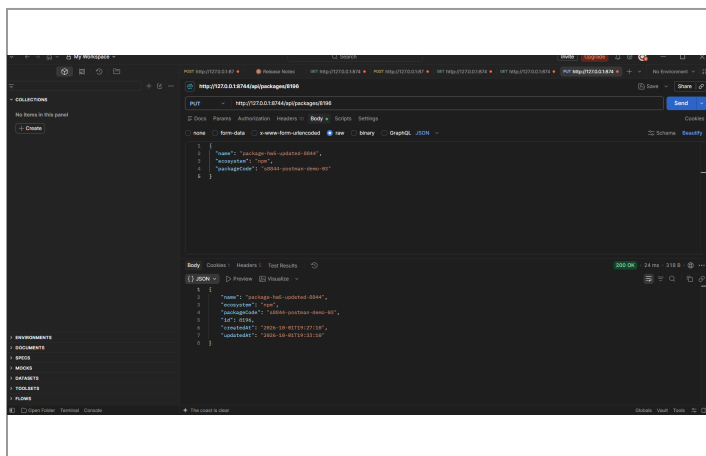
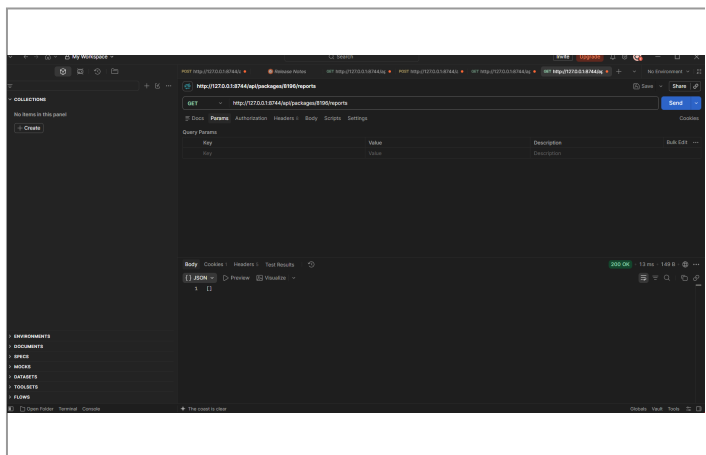
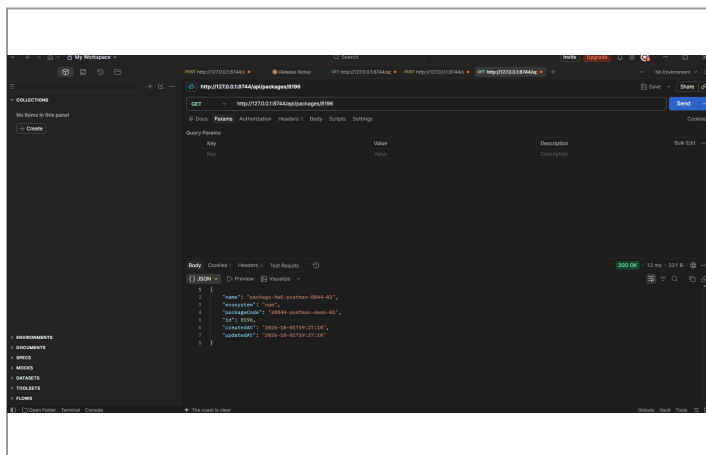
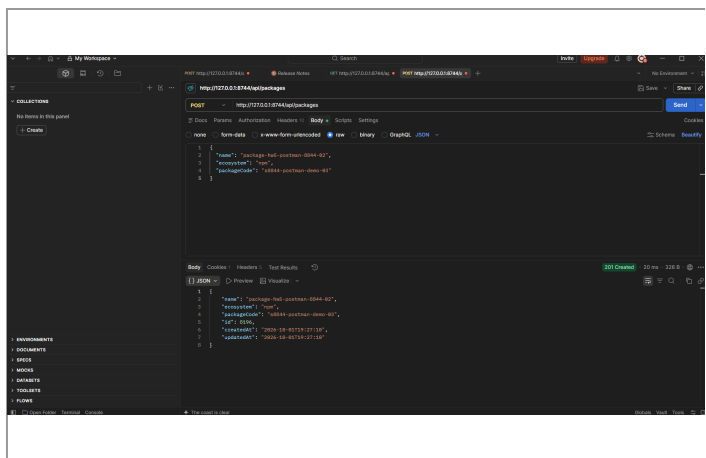
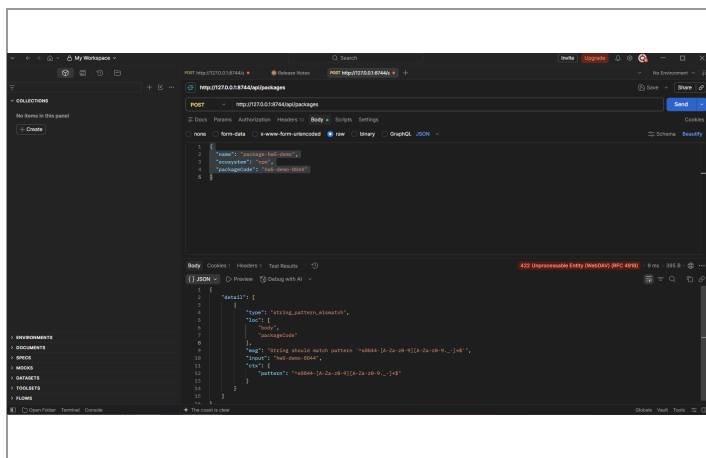


Figure 6: Frontend build success





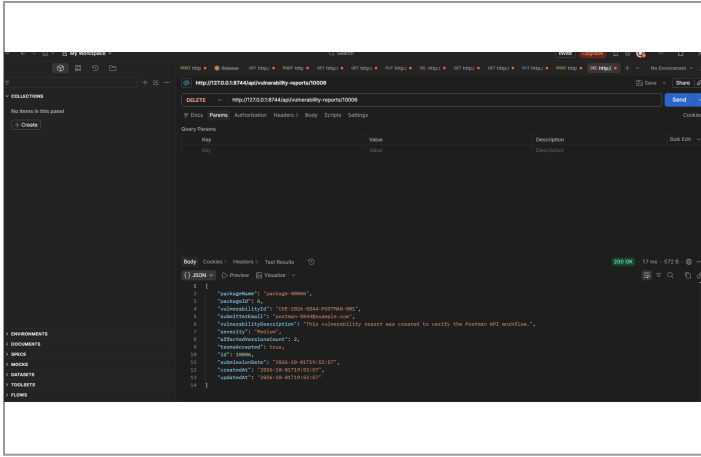


Figure 19: Report deletion

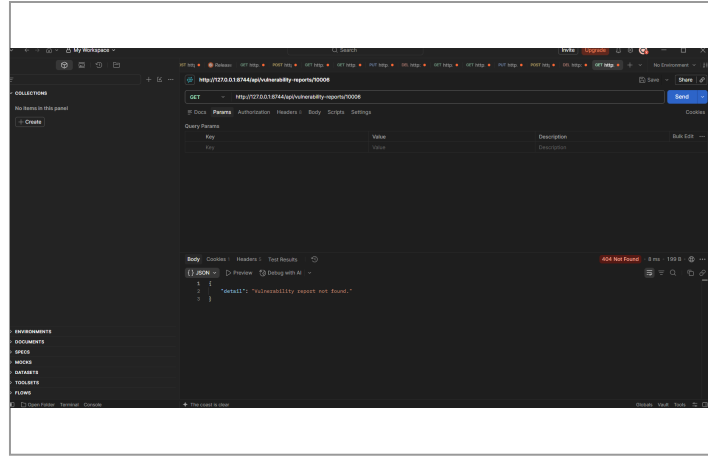


Figure 20: Deleted report 404

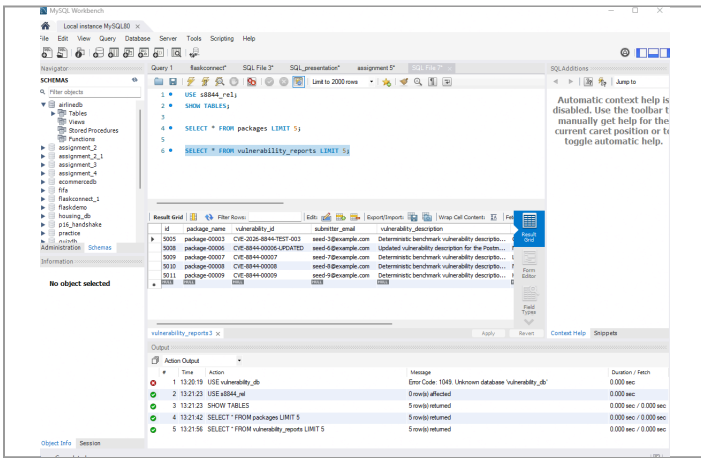


Figure 21: Database evidence

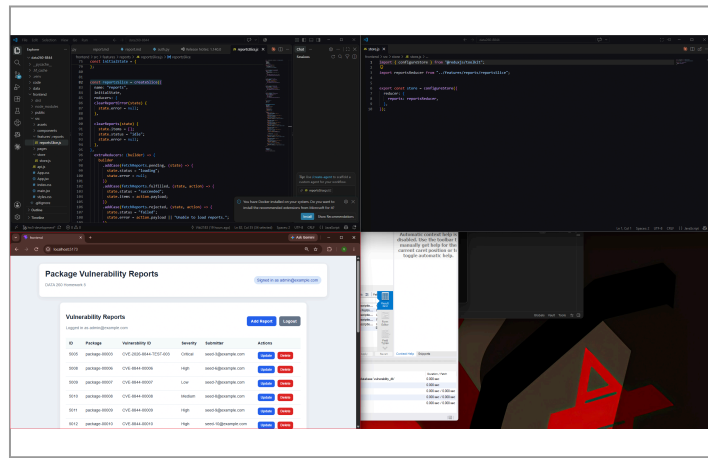


Figure 22: Redux Home code and UI

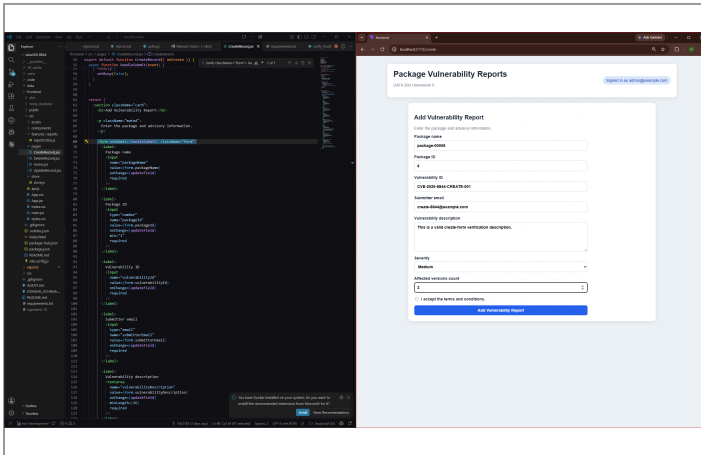


Figure 23: Redux Create code and UI

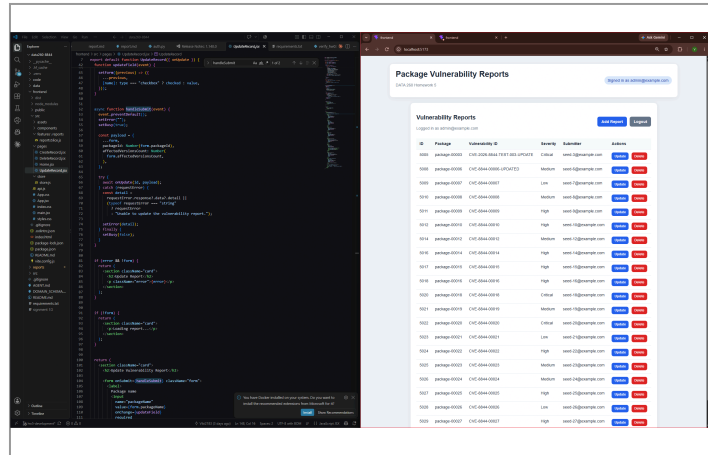


Figure 24: Redux Update code and UI

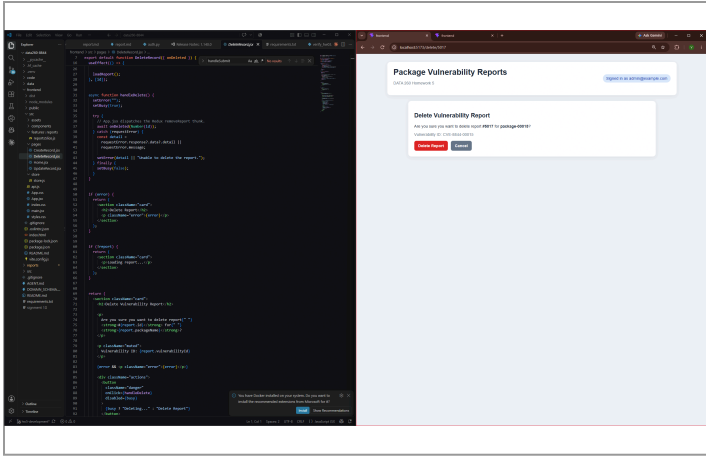


Figure 25: Redux Delete code and UI

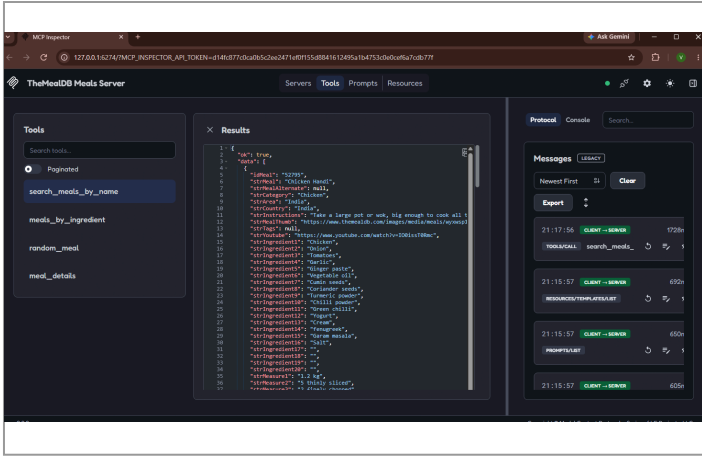


Figure 26: Meal search by name

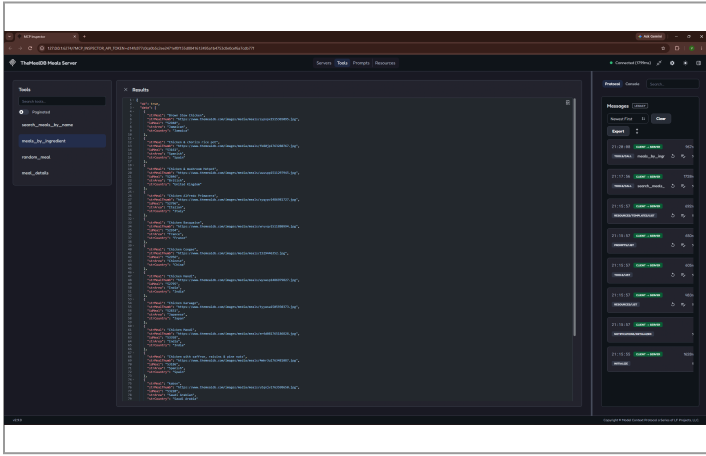


Figure 27: Meal search by ingredient

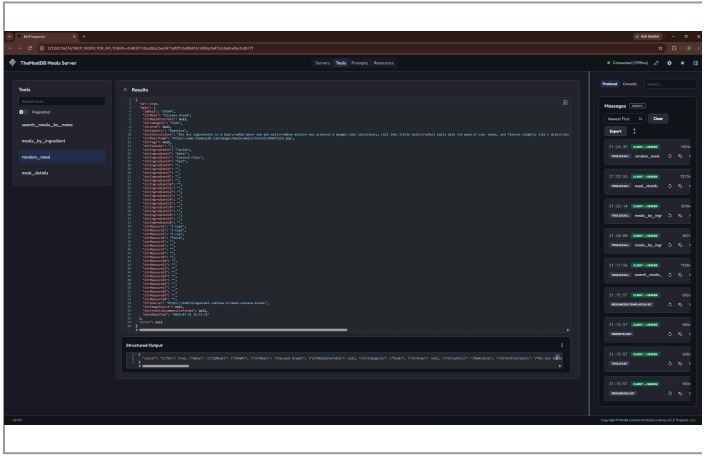


Figure 28: Random meal

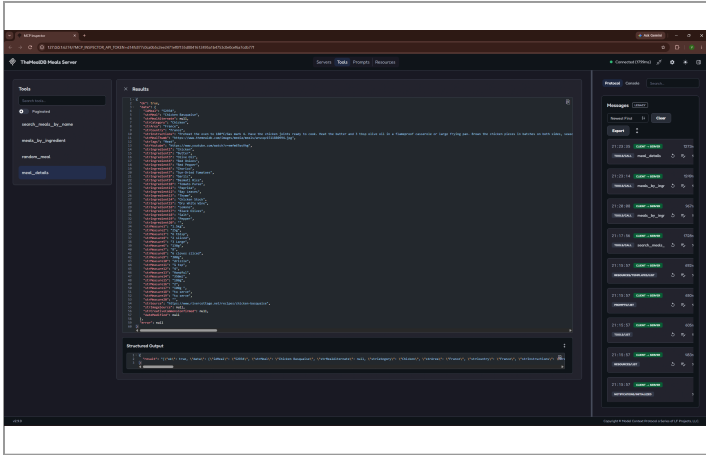


Figure 29: Meal details

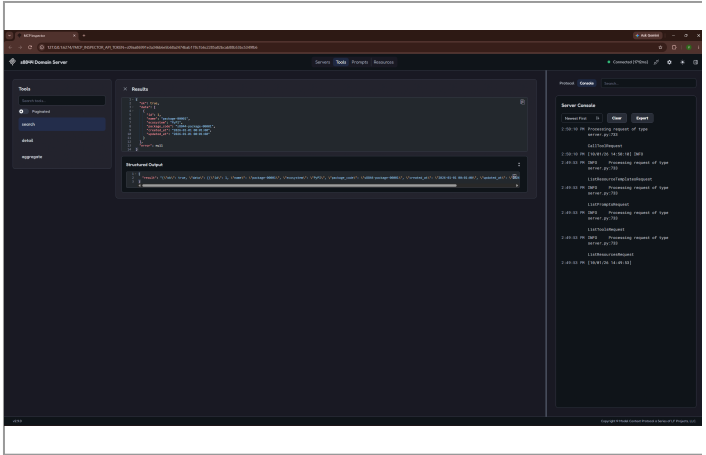


Figure 30: Domain search success

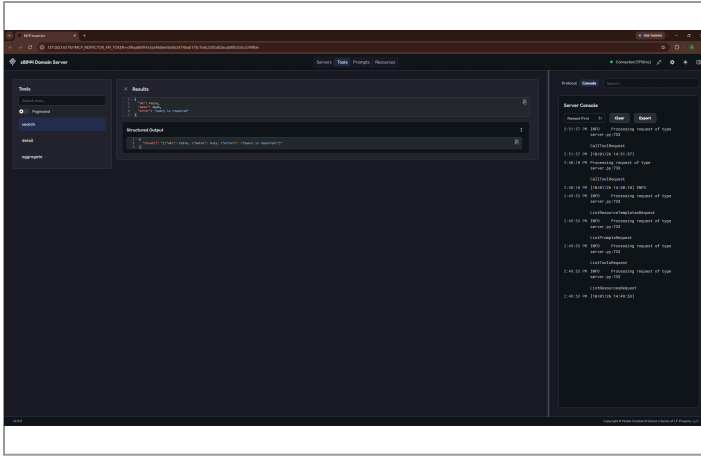


Figure 31: Domain search invalid input

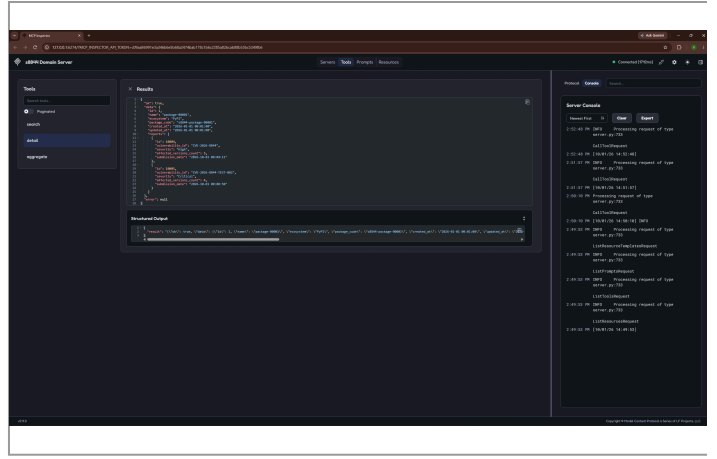


Figure 32: Domain detail success

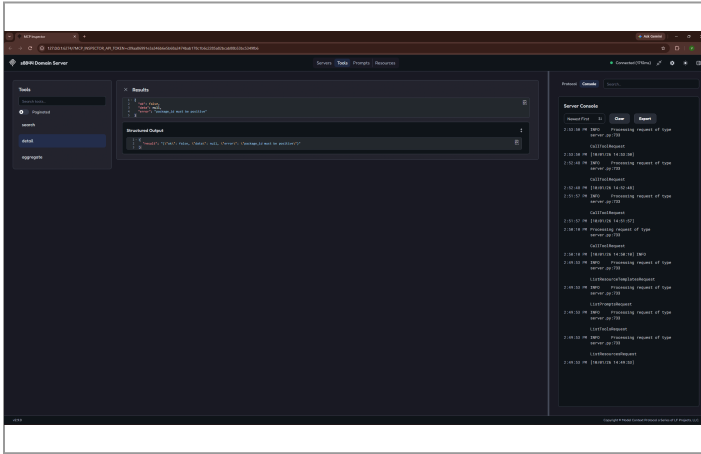


Figure 33: Domain detail invalid input

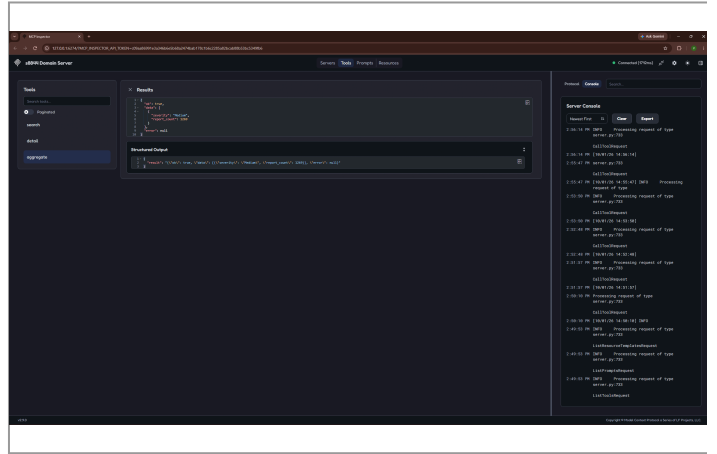


Figure 34: Domain aggregate success

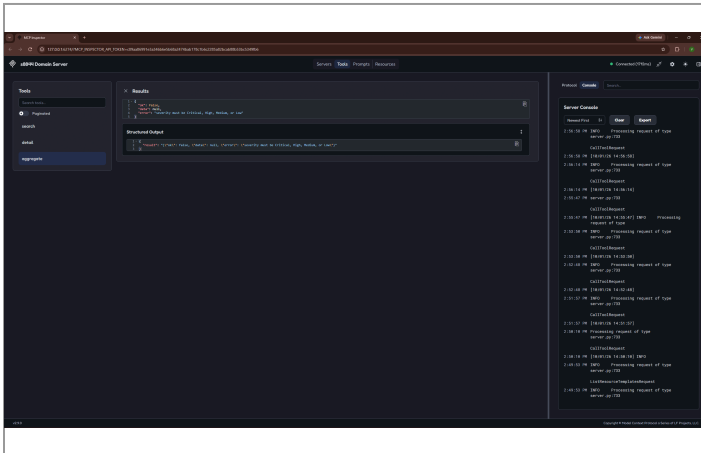


Figure 35: Domain aggregate invalid input

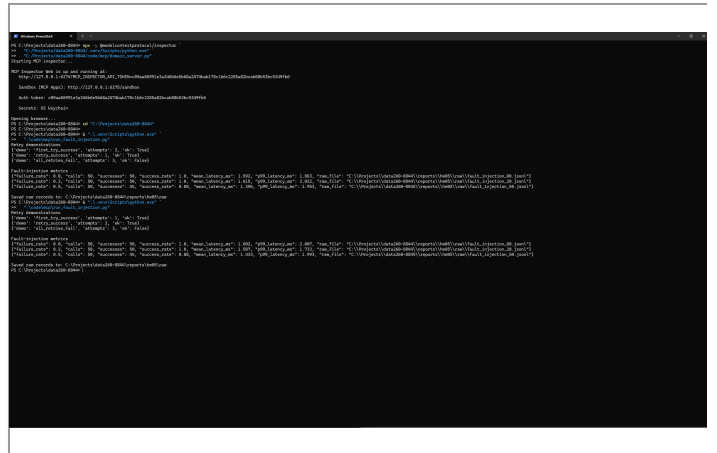


Figure 36: Retry and fault injection

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Vrishin Dharwesh KP> cd "C:\Projects\data260-8844"
PS C:\Projects\data260-8844>
PS C:\Projects\data260-8844> & ".\code\wcp\test_execute_tool.py"
ASS search valid
ASS search invalid
ASS detail valid
ASS detail invalid
ASS aggregate valid
ASS aggregate invalid
ASS unknown tool rejected
SUMMARY: 7/7 tests passed
PS C:\Projects\data260-8844>
```

Figure 37: Execute-tool tests

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Vrishin Dharwesh KP> cd "C:\Projects\data260-8844"
PS C:\Projects\data260-8844>
PS C:\Projects\data260-8844> & ".\code\wcp\test_execute_tool.py"
ASS search valid
ASS search invalid
ASS detail valid
ASS detail invalid
ASS aggregate valid
ASS aggregate invalid
ASS unknown tool rejected
SUMMARY: 7/7 tests passed
PS C:\Projects\data260-8844> cd "C:\Projects\data260-8844"
PS C:\Projects\data260-8844>
PS C:\Projects\data260-8844> & ".\code\wcp\agent.py" --tests
ASS safety allowed call
ASS safety blocked call
ASS MockModel normal completion
ASS MockModel max_steps
SUMMARY: 4/4 agent tests passed
PS C:\Projects\data260-8844>
```

Figure 38: Agent safety tests

```
saved metrics to: C:\Projects\data260-8844\reports\hw05\raw\agent_scenarios_metrics.json
agent runs are logged to:
C:\Projects\data260-8844\reports\hw05\raw\agent_runs.jsonl
PS C:\Projects\data260-8844> Get-Content '
C:\Projects\data260-8844\reports\hw05\raw\agent_runs.jsonl' |
Select-Object -Last 12
[user_input": "Find the package.", "step": 2, "action": "final", "final_answer": "The package search completed.", "stop_reason": "model_f
nal", "trace": [{"step": 1, "tool": "search", "inputs": {"query": "package"}, "result": {"ok": true, "data": [{"name": "fixture-package"}
}, "error": null}]]], [{"step": 2, "tool": "search", "inputs": {"query": "package"}, "result": {"ok": true, "data": [{"name": "fixture-pack
age"}], "error": null}]]], [{"step": 2, "action": "final", "final_answer": "The maximum step limit was reached.", "stop_reason": "max
steps", "trace": [{"step": 1, "tool": "search", "inputs": {"query": "package"}, "result": {"ok": true, "data": [{"name": "fixture-package"}
}, "error": null}]]], [{"step": 2, "tool": "search", "inputs": {"query": "package"}, "result": {"ok": true, "data": [{"name": "fixture-pack
age"}], "error": null}]]], [{"user_input": "Count the vulnerability reports with Medium severity.", "step": 2, "action": "final", "final_answer": "There are 1269 vuln
erability reports with Medium severity.", "stop_reason": "model_final", "trace": [{"step": 1, "tool": "aggregate", "inputs": {"severity":
"Medium"}, "result": {"ok": true, "data": [{"severity": "Medium", "report_count": 1269}], "error": null}]]], [{"user_input": "Count the vulnerability reports with Medium severity.", "step": 1, "action": "final", "final_answer": "Invalid model actio
n.", "stop_reason": "invalid_model_action", "trace": []}], [{"user_input": "Find packages matching package-00001.", "step": 2, "action": "final", "final_answer": "Found package: package-00001 with I
D 1.", "stop_reason": "model_final", "trace": [{"step": 1, "tool": "search", "inputs": {"query": "package-00001", "limit": 5}, "result": {
"ok": true, "data": [{"id": 1, "name": "package-00001", "ecosystem": "DyPI", "package_code": "s8844-package-00001", "created_at": "2026-01
-01 00:01:00", "updated_at": "2026-01-01 00:01:00"}], "error": null}]]], [{"user_input": "Show the details for package ID 1.", "step": 2, "action": "final", "final_answer": "Package details for ID 1:\n\n **ID*:
1\n **Name*: package-00001\n **Ecosystem*: DyPI\n **Package Code*: s8844-package-00001\n **Created At*: 2026-01-01 00:01:00\n **
Updated At*: 2026-01-01 00:01:00\n\n**Reports*\n\n **Vulnerability ID*: CVE-2026-8844-TEST-002, **Severity*: High, **Affected Versions Count*:
5, **Submission Date*: 2026-10-01 00:04:11\n2. **Vulnerability ID*: CVE-2026-8844-TEST-002, **Severity*: Critical, **Affected Version
Count*: 4, **Submission Date*: 2026-10-01 00:08:50", "stop_reason": "model_final", "trace": [{"step": 1, "tool": "detail", "inputs": {
"package_id": 1}, "result": {"ok": true, "data": {"id": 1, "name": "package-00001", "ecosystem": "DyPI", "package_code": "s8844-package-00
001", "created_at": "2026-01-01 00:01:00", "updated_at": "2026-01-01 00:01:00", "reports": [{"id": 10004, "vulnerability_id": "CVE-2026-88
44-TEST-002", "severity": "High", "affected_versions_count": 5, "submission_date": "2026-10-01 00:04:11"}, {"id": 10005, "vulnerability_id": "CVE-2
026-8844-TEST-002", "severity": "Critical", "affected_versions_count": 4, "submission_date": "2026-10-01 00:08:50"}], "error": null}]]], [{"user_input": "Count the vulnerability reports with Medium severity.", "step": 2, "action": "final", "final_answer": "There are 1269 vuln
erability reports with Medium severity.", "stop_reason": "model_final", "trace": [{"step": 1, "tool": "aggregate", "inputs": {"severity":
"Medium"}, "result": {"ok": true, "data": [{"severity": "Medium", "report_count": 1269}], "error": null}]]], [{"user_input": "Search for password records.", "step": 2, "action": "final", "final_answer": "The search for password records was blocked
due to a safety rule preventing credential-like searches.", "stop_reason": "model_final", "trace": [{"step": 1, "tool": "search", "inputs":
{"query": "password", "limit": 5}, "result": {"ok": false, "data": null, "error": "safety rule blocked credential-like search"}]]]
PS C:\Projects\data260-8844>
```

Figure 39: Ollama scenarios

# Submission Files

- RUN\_LOG.txt
- METRICS.md
- AI\_USE.md
- REFLECTION.md
- verification.json
- raw/agent\_runs.jsonl
- raw/agent\_scenarios\_metrics.json
- raw/fault\_injection\_00.jsonl
- raw/fault\_injection\_20.jsonl
- raw/fault\_injection\_50.jsonl
- raw/fault\_injection\_metrics.json
- report.pdf